Topological Signatures of Human Activity

Persistent homology of delay-embedded smartphone sensors

SF2956 Project Group

September 2026

Abstract

We study whether the topology of short acceleration and gyroscope trajectories distinguishes six activities for people not used during model fitting. Each 2.56-second sensor window is mapped to a normalized delay-coordinate point cloud. Vietoris–Rips persistence in H_0 and H_1 is represented by four stable-rank functions, and the longest persistent H_1 class is additionally converted into a circular motion coordinate. Under one fixed subject-grouped evaluation protocol, delay topology obtains 0.580 balanced accuracy on nine disjoint test subjects, compared with 0.499 after time shuffling and 0.558 for unordered instantaneous topology. Restoring two discarded physical-scale values raises the score to 0.639. A compact 16-dimensional persistence-and-dynamics representation obtains 0.568, while the supplied 561 engineered features obtain 0.954. The topological construction therefore provides an interpretable description of temporal motion geometry, but it is not the strongest activity classifier.

1 Introduction

A smartphone sensor window is a short multivariate path, not merely an unordered sample from $\mathbb{R}^3$. Walking and static posture can differ in amplitude, but also in the geometry traced through time. We represent this geometry using delay coordinates and then follow the course pipeline from a finite metric space to Vietoris–Rips persistence and stable ranks [1].

The project asks one primary question:

> Does the topology of short acceleration and gyroscope trajectories contain activity information that transfers to people not used for training?

Two secondary questions make the result interpretable. First, does any information disappear when temporal order is destroyed? Second, can a persistent loop be converted into a circular coordinate describing periodic motion? The objective is not to outperform engineered activity-recognition features. It is to identify what this topological representation measures and where information is lost.

Earlier versions of the project accumulated many classifiers, parameter searches, metric couplings, and statistical comparisons. The present analysis deliberately contains five topological representations, one fixed model, one grouped development protocol, and one official subject-disjoint evaluation. Since earlier project iterations had already inspected the official test results, those results are descriptive rather than a new confirmatory test.

2 Data and sampling structure

The UCI Human Activity Recognition Using Smartphones dataset contains 30 volunteers performing walking, walking upstairs, walking downstairs, sitting, standing, and laying [2]. Triaxial body acceleration and angular velocity were sampled at 50 Hz and divided into 128-sample windows with 50% overlap.

The supplied split assigns complete people to development or test data. This avoids the direct leakage that would occur if overlapping windows from one person appeared on both sides of a random window-level split.

Table 1: Verified structure of the supplied UCI HAR split.

Split	Windows	Subjects	Walk	Up	Down	Sit	Stand	Lay
Development	7,352	21	1,226	1,073	986	1,286	1,374	1,407
Official test	2,947	9	496	471	420	491	532	537

The unit of analysis is one labelled window. For sensor $s \in \{a, g\}$, representing body acceleration and gyroscope respectively, a window is a path

$$x^{(s)} : \{0, \ldots, 127\} \longrightarrow \mathbb{R}^3.$$

3 Topological construction

3.1 Delay-coordinate metric spaces

For embedding dimension m and lag ℓ, define

$$E_{m,\ell}(x)_t = \big(x_t, x_{t+\ell}, \ldots, x_{t+(m-1)\ell}\big) \in \mathbb{R}^{3m}$$

for $t = 0, \ldots, 127 - (m-1)\ell$. The representing finite set is

$$X_{m,\ell}(x) = \{E_{m,\ell}(x)_t\}.$$

Delay coordinates are motivated by reconstruction ideas from dynamical systems [3], but no exact Takens reconstruction claim is made for these finite, noisy and preprocessed paths.

We fix $m = 3$ and $\ell = 15$, producing 98 points in $\mathbb{R}^9$. The lag is the first non-positive value of the mean normalized acceleration autocorrelation on the development data (Figure 1); it is not selected using classification accuracy.

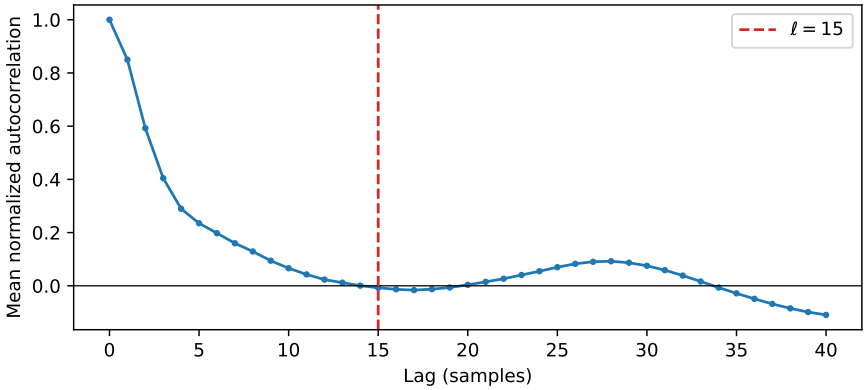


Figure 1: Mean development-set acceleration autocorrelation. The first non-positive value occurs at 15 samples, or 0.30 seconds.

For a cloud $X = \{z_1, \ldots, z_n\}$, define

$$\bar{z} = \frac{1}{n}\sum_i z_i, \qquad r(X) = \left(\frac{1}{n}\sum_i \|z_i - \bar{z}\|_2^2\right)^{1/2},$$

and normalize by

$$\widetilde{X} = \left\{\frac{z_i - \bar{z}}{r(X)}\right\}_{i=1}^{n}.$$

Thus the primary topology is invariant to translation and positive global rescaling. The two sensor radii are retained separately for one explicit comparison rather than being silently mixed into the normalized topology.

3.2 Vietoris–Rips persistence

For finite metric space (X, d) and scale $\epsilon \geq 0$, the Vietoris–Rips complex is

$$\mathrm{VR}_\epsilon(X) = \{\sigma \subseteq X : d(x, y) \leq \epsilon \text{ for every } x, y \in \sigma\}.$$

Because X is finite, the filtration is tame [1, Points 187–193]. We compute persistence diagrams $D_0(X)$ and $D_1(X)$ over $\mathbb{F}_{47}$ using Ripser.py [4]. Here H_0 records how connected components merge and H_1 records loops. The field is shared with the circular-coordinate extension so that one persistence call supplies both representations.

Figure 2 shows one walking acceleration window. Only the first three of nine delay coordinates are drawn; persistence is computed from the full Euclidean geometry.

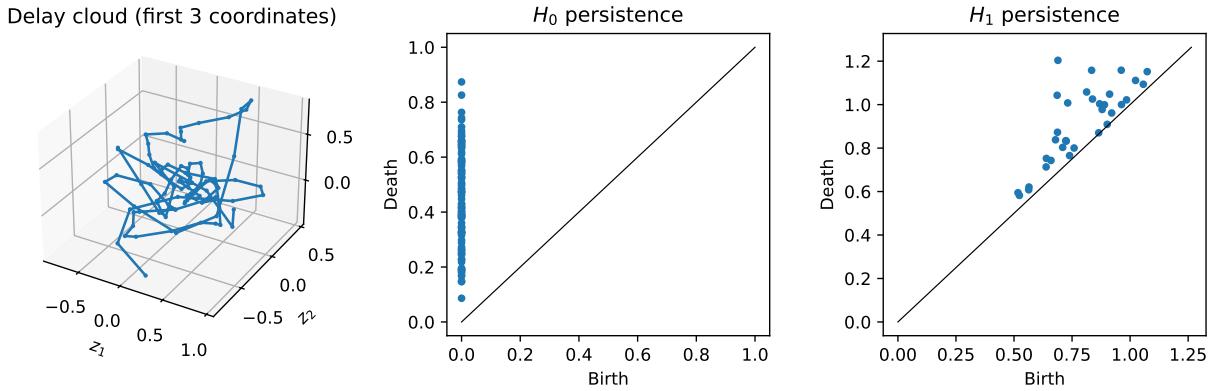


Figure 2: A normalized delay cloud and its finite H_0 and H_1 persistence diagrams for one development window.

3.3 Stable ranks

For finite diagram D_k, the lifetime stable rank is

$$\rho_k(u) = \#\{(b,d) \in D_k : d - b > u\}, \qquad u \geq 0.$$

We evaluate this function with StableBear [5, 6] on the fixed grid $u_j = 2j/63$, $j = 0, \ldots, 63$. One window becomes

$$\Phi_{\mathrm{SR}}(x) = [\rho_0^{(a)}, \rho_1^{(a)}, \rho_0^{(g)}, \rho_1^{(g)}] \in \mathbb{R}^{256}.$$

Infinite H_0 classes are removed. The blockwise numerical distance used later is

$$d_{\mathrm{SR}}(f,g) = \sum_{b=1}^{4} \frac{2}{63} \sum_{j=0}^{62} |f_b(u_j) - g_b(u_j)|.$$

Computing the integral separately for each block is important: each block has its own right endpoint.

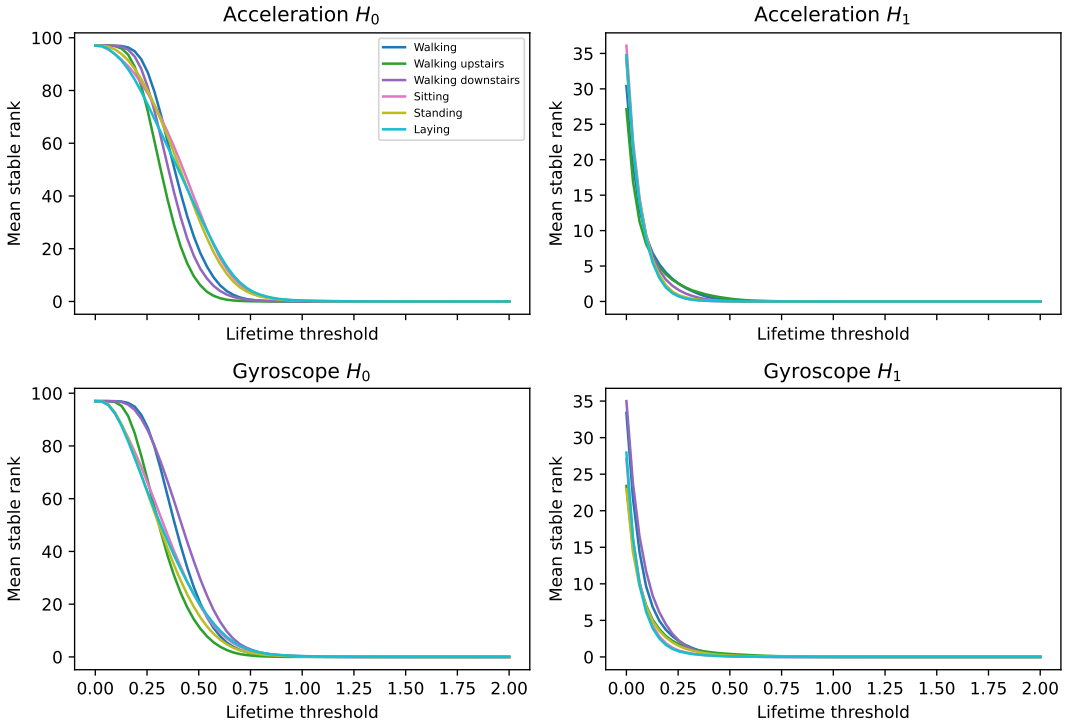


Figure 3: Mean stable ranks by activity. Dynamic and static activities differ broadly, while similar activities overlap.

3.4 Persistent circular coordinates

Stable ranks count lifetimes but discard the birth scale and identity of a particular loop. We therefore retain six direct summaries per sensor: mean and maximum finite H_0 lifetime, number of H_1 classes, longest H_1 lifetime, total H_1 persistence, and birth of the longest class.

For the longest H_1 bar (b^*, d^*), Ripser also returns a cocycle α over $\mathbb{F}_{47}$. We evaluate it on the Rips graph at

$$\epsilon^* = b^* + \tfrac{1}{2}(d^* - b^*)$$

and lift its coefficients to centered integers. If B is the oriented edge–vertex incidence matrix, fix the first vertex and solve

$$\widehat{\phi} = \arg\min_{\phi:\phi_0=0} \|B\phi + \alpha\|_2^2.$$

The circle-valued coordinate is $\theta_i = \widehat{\phi}_i \bmod 1$ [7]. Unwrapping $2\pi\theta_i$ in delay-point order and regressing it on $t_i = i/50$ yields circular frequency $|\widehat{\beta}_1|/(2\pi)$ and phase linearity R^2. Together with the six diagram summaries, this gives eight features per sensor and 16 per window.

4 Experimental design

The comparison contains exactly five topological representations:

1. instantaneous TDA: $m = 1, \ell = 1$;
2. delay TDA: $m = 3, \ell = 15$;
3. shuffled-delay TDA: independently permute each window before the same delay construction;
4. delay TDA plus the two sensor radii;
5. the 16 compact persistence-and-dynamics features.

The shuffled control preserves the multiset of three-dimensional samples while destroying temporal adjacency. The supplied 561 UCI features are included once as a non-topological reference.

Every representation uses the same pipeline:

$$\text{StandardScaler} \longrightarrow \text{RBF SVM}(C = 10, \gamma = 0.001, \text{class-weight=balanced}).$$

The parameters are frozen from earlier development work. We perform no representation-specific tuning. Development predictions use five stratified subject-grouped folds with seed 2956; the fitted model is then evaluated once on the supplied disjoint test subjects. Balanced accuracy is primary because activity counts differ, and macro F1 is secondary. Scikit-learn supplies the model and grouped folds [8].

5 Results

5.1 Subject-independent classification

Table 2: One fixed evaluation protocol for every representation. BA denotes balanced accuracy.

Representation	Development BA	Test BA	Test macro F1
Instantaneous TDA	0.525	0.558	0.553
Delay TDA	0.533	0.580	0.576
Shuffled-delay TDA	0.482	0.499	0.483
Delay TDA + scale	0.621	0.639	0.640
Topological dynamics	0.516	0.568	0.564
UCI 561 baseline	0.938	0.954	0.955

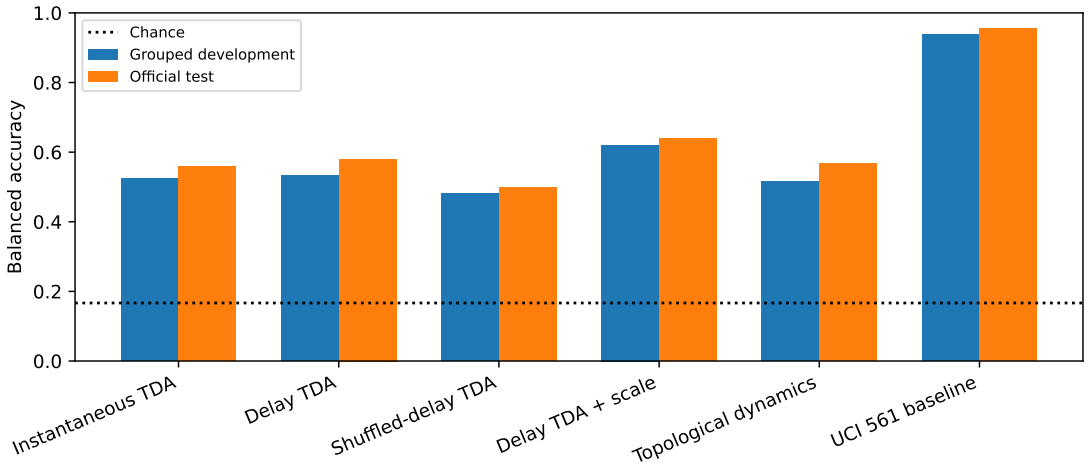


Figure 4: Grouped development and subject-disjoint test balanced accuracy under the shared fixed model.

Delay topology exceeds shuffled-delay topology by 0.081 test balanced-accuracy points. This supports the claim that temporal adjacency contributes to the delay-cloud topology. Delay also exceeds instantaneous topology by 0.022, but this smaller descriptive difference alone is not strong evidence that delay embedding is always preferable.

Restoring only the two RMS radii raises the delay score from 0.580 to 0.639. Normalization therefore removes physical amplitude that is useful for activity recognition. This is not a defect in the construction: it exposes the distinction between normalized shape and scale.

The 16-dimensional dynamics vector reaches 0.568. It is competitive with the 256-dimensional stable-rank vector but does not exceed it under the shared fixed classifier. The earlier, higher dynamics result depended on representation-specific tuning; removing that tuning changes the conclusion. The engineered UCI features remain substantially stronger at 0.954.

5.2 Cross-subject stable-rank geometry

For development subject p and activity c, define

$$\mu_{p,c} = \frac{1}{N_{p,c}} \sum_{i:p_i=p,y_i=c} \Phi_{\mathrm{SR}}(x_i).$$

Figure 5 averages $d_{\mathrm{SR}}(\mu_{p,c}, \mu_{q,c'})$ across distinct subjects. The mean diagonal distance, comparing the same activity across people, is 13.75. The mean off-diagonal distance is 18.87, a ratio of 1.37. Walking is closest to walking downstairs, and sitting is closest to standing. The geometry therefore recovers a broad dynamic–static division without another classifier, while also showing substantial subject variation.

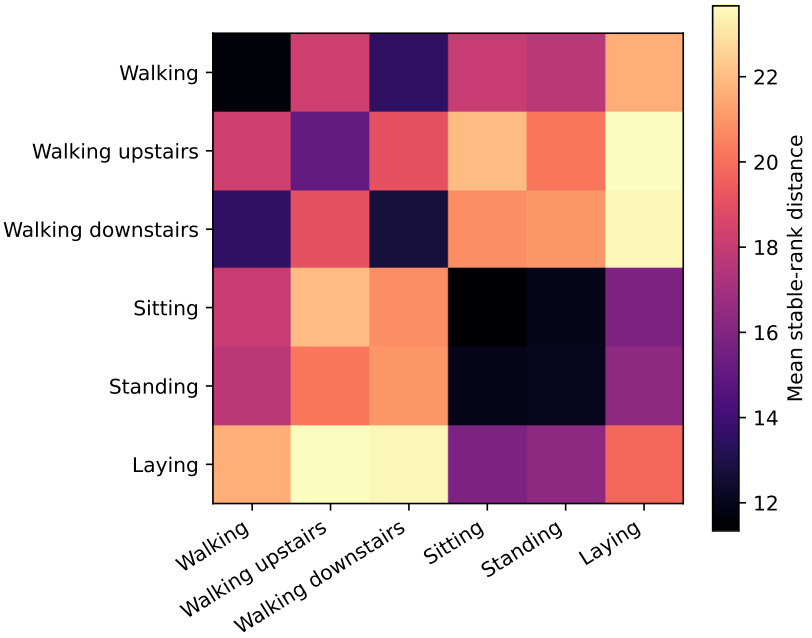


Figure 5: Mean blockwise stable-rank distance between subject–activity prototypes.

5.3 Circular motion summaries

Figure 6 shows four directly interpretable components of the compact representation. The dynamic activities have higher mean circular frequency than sitting, standing and laying. Phase linearity is also generally higher for periodic walking motion. These quantities describe the phase of a selected persistent loop; they are not identical to a Fourier peak and should not be interpreted as a complete physical gait frequency.

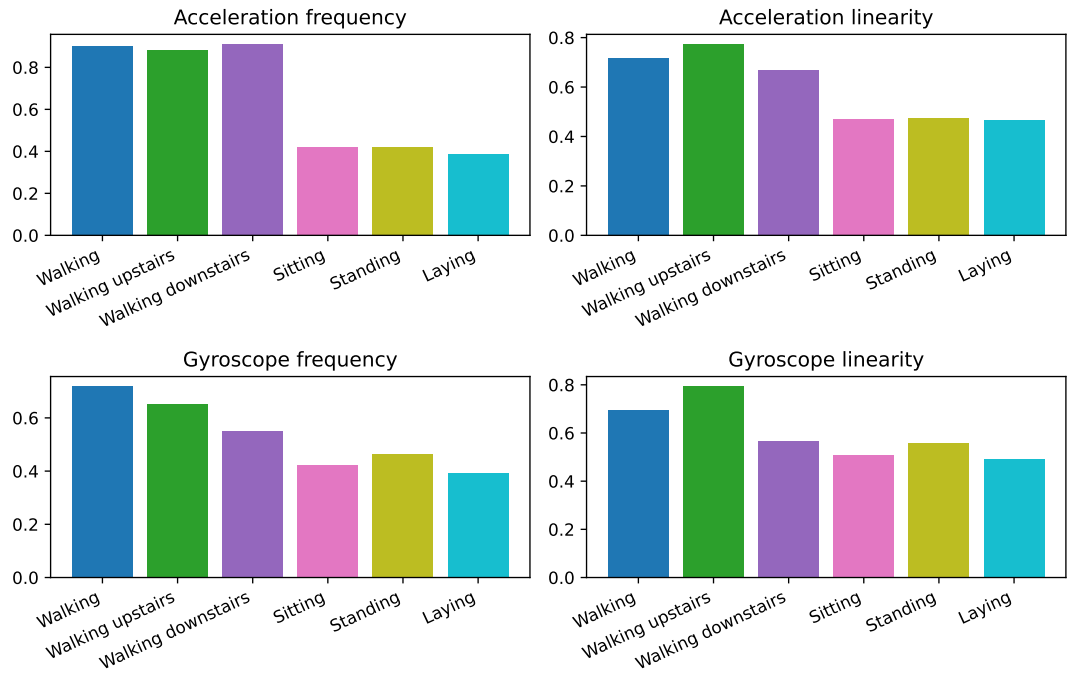


Figure 6: Activity means for circular frequency and phase linearity from the longest persistent H_1 class.

5.4 Bounded perturbation check

We select ten development windows per activity and add independent Gaussian noise with standard deviation equal to 5% of each window and channel's empirical standard deviation. After complete recomputation, the mean stable-rank displacement is 2.61, with empirical 2.5% and 97.5% quantiles 1.86 and 4.19. This is about 19% of the mean same-activity cross-subject distance. The result illustrates gradual movement under perturbation, consistent with the Rips stability mechanism in the course notes [1, Point 197]; it is not a proof of a stability theorem.

6 Discussion

The clean analysis supports three main conclusions. First, normalized delay topology contains activity information that transfers between subjects. Second, destroying temporal adjacency reduces performance, so the representation is not merely detecting an unordered distribution of samples. Third, normalization deliberately removes important amplitude information, visible in the improvement from adding two explicit radii.

The topological representation is most useful as an interpretable structural description. The stable-rank geometry places dynamic and static activities into broad families, while the circular coordinate connects a persistent loop to ordered phase progression. These are meaningful TDA results even though ordinary engineered features classify much better.

There are important limitations. The official test was inspected during earlier project iterations, so its scores are descriptive. The fixed SVM parameters also originate from that exploratory work. Windows overlap within subjects, although subject-grouped validation prevents a person from appearing in both fitting and validation folds. Stable ranks discard bar births and correspondence between classes. Finally, a circular coordinate derived from the longest H_1 class can be unstable when several loops have similar persistence or when no clear periodic class exists.

The deliberately omitted analyses are also informative. Large parameter grids, multiple sensor-coupling metrics, extensive ablations and repeated uncertainty calculations made the previous implementation difficult to audit without changing the core conclusion. They are not required to demonstrate the course pipeline. Future work should begin with new subjects or a genuinely untouched dataset rather than expanding the current search space.

7 Reproducibility

The analysis is specified mathematically in `ANALYSIS_SPEC.md`. Four small Python files load the data, construct topological features, evaluate one fixed model, and run the analysis. The primary delay persistence is computed once per window and reused for stable ranks and circular dynamics. Direct compressed NumPy caches avoid recomputing persistence; fitted models are not serialized.

From the project root:

```
.venv/bin/python -m unittest -v test_math.py
.venv/bin/python run_analysis.py --smoke --recompute
.venv/bin/python run_analysis.py
.venv/bin/python build_notebook.py
```

The cold full run completed in 8 minutes 33 seconds on the development machine. The smoke run completed in approximately ten seconds. All nine mathematical and data-contract tests pass, and the generated notebook executes all ten code cells from a fresh kernel.

8 Conclusion

Delay-embedded smartphone sensors produce persistent topological structure that generalizes across people. Time shuffling lowers stable-rank classification, while adding back two explicit physical scales improves it. Persistent circular coordinates provide a compact description of periodic motion but, without representation-specific tuning, do not outperform the stable-rank vector. The supplied engineered features remain far stronger. The contribution is therefore a small, auditable TDA account of motion geometry and temporal structure rather than a claim of state-of-the-art activity recognition.

References

[1] W. Chacholski. *Analysis pipeline: SF2956 Topological Data Analysis course notes.* KTH Royal Institute of Technology.

[2] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In *ESANN*, 2013.

[3] F. Takens. Detecting strange attractors in turbulence. In *Dynamical Systems and Turbulence, Warwick 1980*, Lecture Notes in Mathematics 898, pp. 366–381, 1981.

[4] C. Tralie, N. Saul, and R. Bar-On. Ripser.py: a lean persistent homology library for Python. *Journal of Open Source Software*, 3(29):925, 2018.

[5] B. Wehlin. StableBear 0.4.3 documentation. https://docs.stablebear.org/en/v0.4.3/.

[6] W. Chachólski and H. Riihimäki. Metrics and stabilization in one parameter persistence. *SIAM Journal on Applied Algebra and Geometry*, 4(1):69–98, 2020.

[7] V. de Silva, D. Morozov, and M. Vejdemo-Johansson. Persistent cohomology and circular coordinates. *Discrete & Computational Geometry*, 45:737–759, 2011.

[8] F. Pedregosa et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.